유승민

백엔드 개발자 (Java / Spring Boot)

yusm1231@gmail.com · GitHub: github.com/yu-teacher

소개

Java와 Spring Boot로 2023년부터 팀·개인 프로젝트 10여 개를 만들었습니다. 도메인 규칙을 프레임워크와 분리해서 테스트로 확인하는 방식을 좋아하고, 외부 서비스(결제, 통관 API, 바둑 엔진)는 포트 뒤로 격리해서 다룹니다. AI 도구를 쓴 프로젝트는 마지막 쪽에 따로 밝혀 두었습니다.

기술 스택

언어/프레임워크 Java 11·21·25, Spring Boot, Spring Security + JWT, OAuth 2.1/OIDC, JPA, QueryDSL
실시간·연동 STOMP/SockJS/WebSocket, gRPC, OpenFeign, 포트원 결제 SDK, Apache POI
DB·인프라 MySQL, PostgreSQL, Redis, H2, Docker, GitHub Actions, AWS(EC2·S3)
테스트·설계 JUnit, 헥사고날 구조, ArchUnit, k6(부하 테스트)

핵심 요약

- 회사 실무 시스템을 직접 만들어 운영했습니다. 기존 담당자가 **하루 5~6시간** 걸리던 주문 처리가 **1시간 내외**로 줄었고, 통합 시스템 이후엔 재고 입력과 출고 지시만 사람이 합니다.
- 도메인 규칙은 테스트와 구조 검사로 지킵니다. 구매대행 **테스트 727개, ArchUnit 규칙 27개** / Doro **688개** / 온라인 바둑 서버 **185개**.
- 인증·인가 플랫폼(Doro)을 설계하고 부하 테스트로 검증했습니다. 로그인 폭주 시 다른 API 성공률 **33%→100%**, 대가(정상 로그인 100%→32%)도 문서화했습니다.

프로젝트 한눈에 보기

프로젝트	쪽	기간	한 줄	내 몫
구매대행 업무 자동화	2	2026-02~09 (이후 유지보수)	주문 변환·검증에서 통합 운영 시스템까지 (회사 업무용)	기획·개발 주도 (2단계는 AI 도구 활용)
주차장 관리 시스템	3	2025-02~06	결제·사용자 서비스 (MSA, 팀)	결제 서비스 100%, 사용자 서비스 일부
Doro	4	2026-09~10	인증·인가·운영 플랫폼	요구·규칙·검증 (AI 도구 협업)
온라인 바둑	5	2025-11~2026-02	바둑 서버 + 채팅 서버 (우아한테크코스 프리코스 최종 과제)	백엔드 2개 100% (프론트는 AI)
KelogBackend	6	2024-04~09	Velog형 블로그 백엔드 (2인)	main 코드 줄의 약 70%
wolMgr	6	2023-03~06	Wake-on-LAN 관리 (4인)	main 줄의 약 77%
team4	6	2023-02	커뮤니티 웹 (6인)	채팅 기능

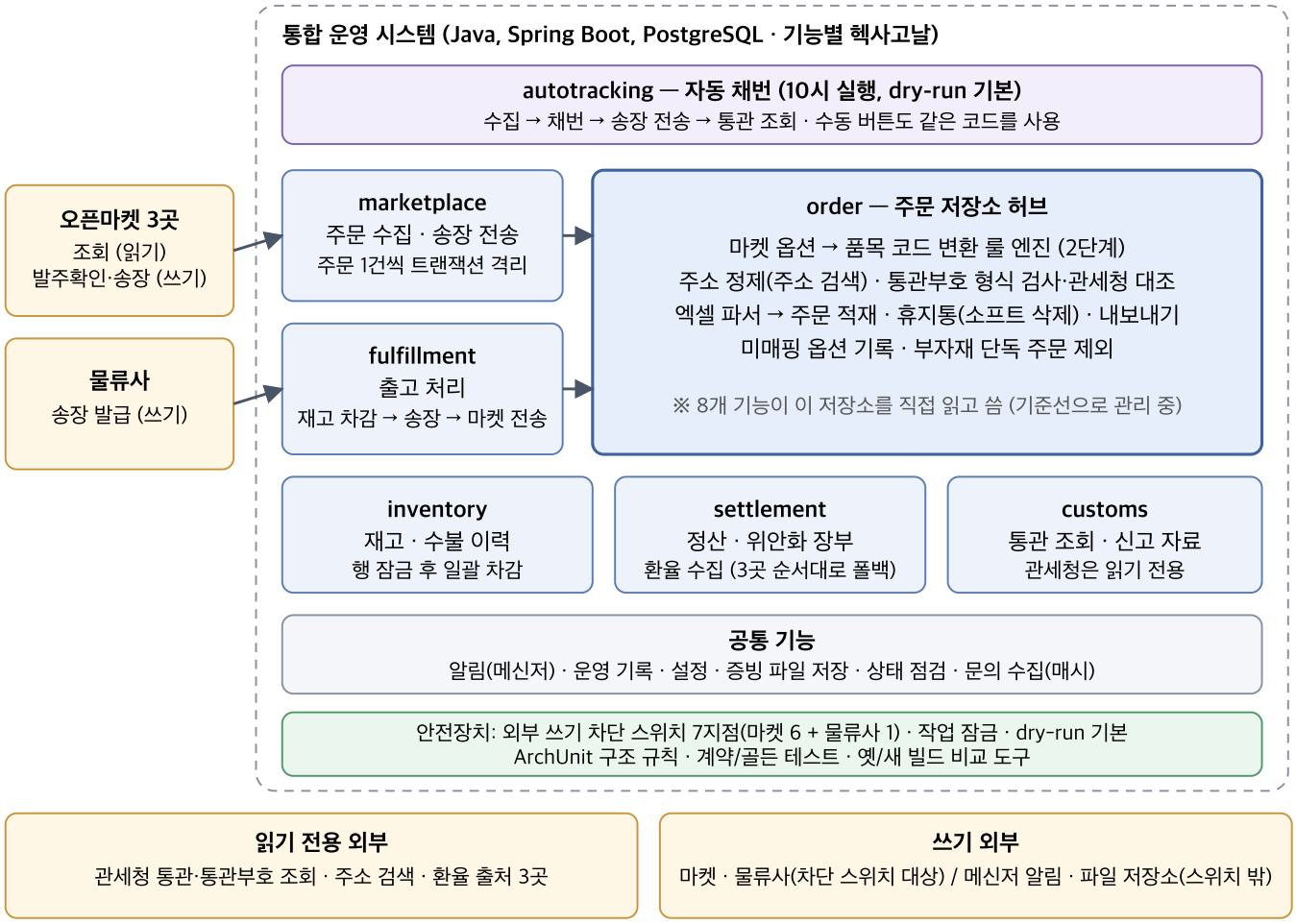
팀 프로젝트 몫은 git blame으로 현재 코드 줄 단위로 계산했습니다.

1. 구매대행 업무 자동화 — order-aggregator에서 통합 운영 시스템까지

2026-02~09 회사 업무로 개발·운영, 이후 개인 유지보수 · Java 21, Spring Boot, PostgreSQL, Apache POI, Docker Compose, GitHub Actions · 1단계는 AI 없이, 2단계(통합 시스템)는 AI 코딩 도구 활용

계기 해외 상품을 사서 국내 오픈마켓에 파는 구매대행 회사에서, 5개 마켓의 서로 다른 주문 엑셀을 송장 출력용 엑셀 하나로 바꾸는 일을 이관받았습니다. 기존 담당 직원이 하루 5~6시간 이상 걸리던 일이었고, 오류는 주로 개인통관고유부호와 주소에서 났습니다(고객 수기 입력, 2025~26년 통관 정책 강화로 예전 부호를 그대로 쓰는 경우). 눈으로 걸러내는 건 비효율이라고 보고 직접 만들기 시작했습니다.

- **1단계 order-aggregator**: 엑셀 형식별 파서를 템플릿 메서드로 공통화하고, 통관부호와 주소는 외부 API로 검증해 유효/무효를 나눠 내려줍니다. 같은 업무가 **하루 1시간 내외**로 줄었습니다. 재고는 버전 관리 없는 엑셀이라 수량이 맞지 않는 일이 반복돼서 구글 스프레드시트를 먼저 도입했습니다.
- **2단계 통합 운영 시스템**: 마켓 주문 API 수집 → 검증 → 물류사 송장 자동 채번 → 마켓 송장 입력, 그리고 재고·정산·통관 자료·알림까지 한 시스템으로 묶었습니다. 재고 입력과 출고 지시서 발송을 빼면 사람이 볼 일이 없어져 **일일 업무 시간이 사실상 없는 수준**이 됐습니다.
- **돈과 재고가 걸려 둔 안전장치**: 외부 쓰기 차단 스위치 7지점, 자동 채번 dry-run 기본, 작업 잠금, ArchUnit 규칙 27개, 계약·골든 테스트, 옛 빌드와 새 빌드의 응답·DB 비교 도구. 테스트 727개.



구매대행 업무 자동화 구조 (main 소스를 전부 읽고 그린, 회사 정보를 제외한 일반화 그림)

파급 데이터 정합성을 본 세무 관련 업체 대표와 자동화 미팅을 했고, 물류사와는 파레트 적재·종합 재고 관리 시스템을 논의해 프로토타입까지 제공했습니다(둘 다 일부만 진행).

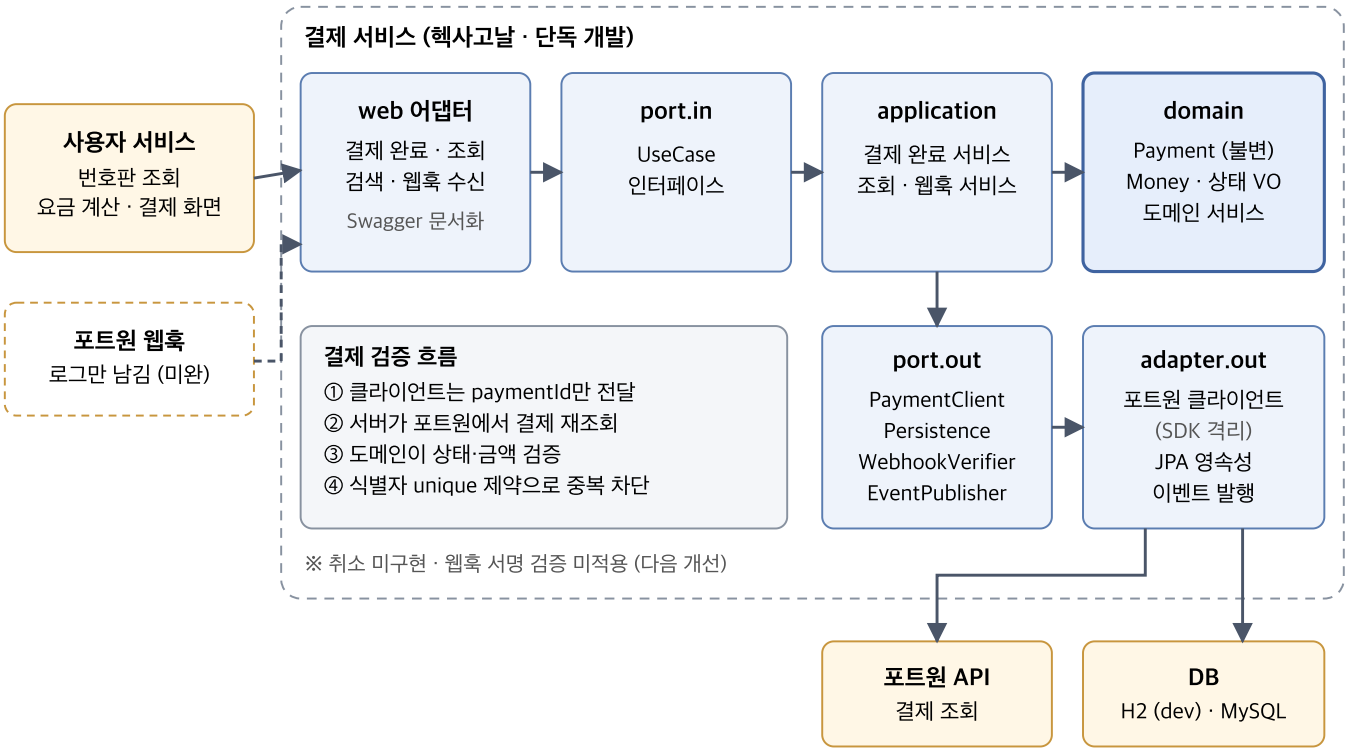
다음에 개선할 것 기능 간에 남은 의존을 단계적으로 줄이고, 외부 쓰기 지점의 차단 누락을 구조적으로 막는 것입니다.

2. 주차장 관리 시스템 — 결제 서비스와 사용자 서비스

2025-02~06 · 팀 프로젝트, 서비스별로 나눈 MSA 구성 · Java 21, Spring Boot 3.4, JPA, 포트원 SDK, OpenFeign, H2/MySQL

맡은 범위 주차장 관리 시스템을 결제 서비스와 사용자 서비스로 나눈 구성에서, **결제 서비스 전체(코드 3,662줄 전부)**를 혼자 만들었고 **사용자 서비스에서는 번호판 검색 → 결제 화면 흐름과 요금 계산**(무료 시간·기본요금·올림 과금)을 맡았습니다. 두 서비스는 OpenFeign으로 연결됩니다.

- **SDK 격리**: 포트원 SDK 타입이 서비스 계층까지 들어와 있어서 결제 대행사를 바꾸거나 테스트하려면 서비스 코드가 함께 흔들리는 구조였습니다. 포트(`port/in·out`)와 어댑터로 나누고 SDK는 어댑터 안에서만 쓰도록 리팩터링했습니다.
- **결제 검증**: 클라이언트는 `paymentId`만 보내고, 서버가 포트원에서 결제를 다시 조회해 상태와 금액을 검증합니다. 클라이언트 값을 믿지 않는 구조이고, 결제 식별자 unique 제약으로 중복 저장을 막습니다.
- **도메인 모델**: `Payment`는 상태를 바꾸면 새 객체를 돌려주는 불변 애그리거트이고, 금액은 `Money` 값 객체로 음수·통화 불일치를 막습니다. 검색은 정렬 필드를 허용 목록으로 제한합니다.



주차장 결제 서비스 구조 (소스를 읽고 그린 그림)

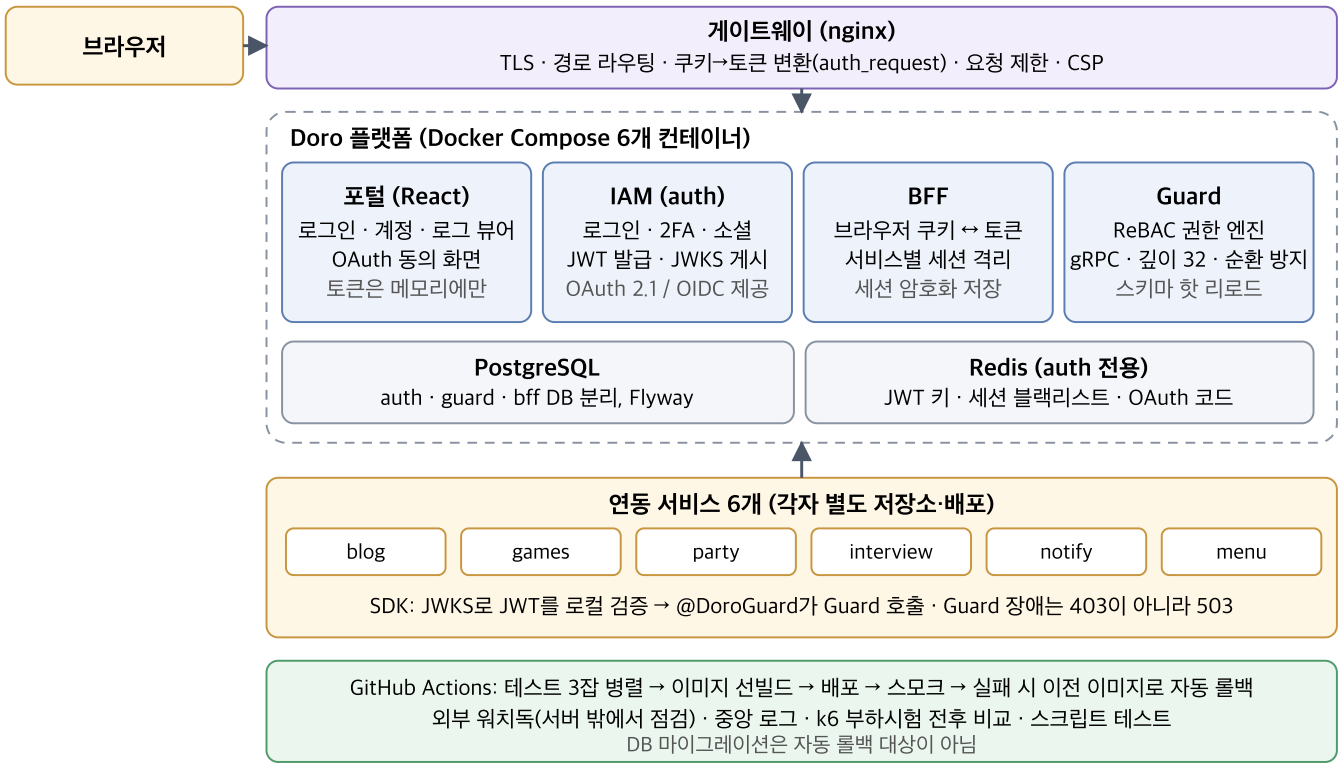
테스트 조회·영속성 27개(`@DataJpaTest`, MockMvc 통합).

다음에 개선할 것 결제 완료 흐름의 트랜잭션 분리, 웹훅 서명 검증, 결제 취소 구현입니다.

3. Doro — 인증·인가와 서비스 운영 플랫폼

2026-09~10 · 1인 개발 · AI 코딩 도구와 협업 · Java 25, Spring Boot 4.1, PostgreSQL, Redis, gRPC, React, nginx, Docker Compose, GitHub Actions

계기 앞으로 서비스를 계속 만들 때, 서비스마다 로그인과 권한을 새로 구현하지 않고 **하나의 계정으로 수십 개의 서비스를 쓸 수 있는 구조**를 먼저 만들어 두려고 시작했습니다. 지금은 블로그, 게임, 파티, 인터뷰, 알림, 메뉴 6개 서비스가 이 플랫폼 뒤에 있고, Java 서비스 5개는 Doro SDK를 의존성으로 씁니다. 요구와 규칙(권한 분기문 하드코딩 금지 등 AI 작업 규칙 문서), 위험 시나리오, 테스트 기준은 제가 정했고 구현은 AI와 함께 했습니다.



Doro 구조 (소스를 전부 읽고 그린 그림, 운영 주소 등은 제외)

- **권한 엔진(Guard)**: 권한을 `if`가 아니라 데이터(관계 튜플과 선언형 스키마)로 분리했습니다. 평가 깊이 제한(32)과 경로별 순환 방지를 두고, 제외 조건이 잘리면 허용하지 않고 거부하며, 잘린 결과는 캐시하지 않습니다. Guard 장애는 "거부"가 아니라 503으로 구분합니다.
- **계정과 토큰(IAM)**: 액세스 JWT 15분과 회전형 리프레시 30일을 발급하고, 폐기된 리프레시가 다시 오면 같은 세션 계열을 모두 폐기합니다. OAuth 2.1/OIDC 제공(PKCE 필수), TOTP, 소셜 로그인을 갖췄고, 브라우저는 게이트웨이가 쿠키를 BFF에서 토큰으로 바꿔 서비스에 붙입니다.
- **측정**: 로그인 폭주 시 해싱이 서버를 잠식하는 문제를 k6로 확인하고 대기열 상한을 두었습니다. 초당 150건 공격 때 `/users/me` 성공률이 33%에서 100%로 올랐고, 대가로 정상 로그인 성공률이 100%에서 32%로 떨어지는 것까지 문서에 적었습니다(로컬 환경의 상대 비교). Java 테스트 688개.
- **운영**: 테스트 3잡 병렬 → 이미지 선빌드 → 배포 → 스모크 → 실패 시 이전 이미지로 자동 롤백, 서버 밖 외부 워치독과 중앙 로그.

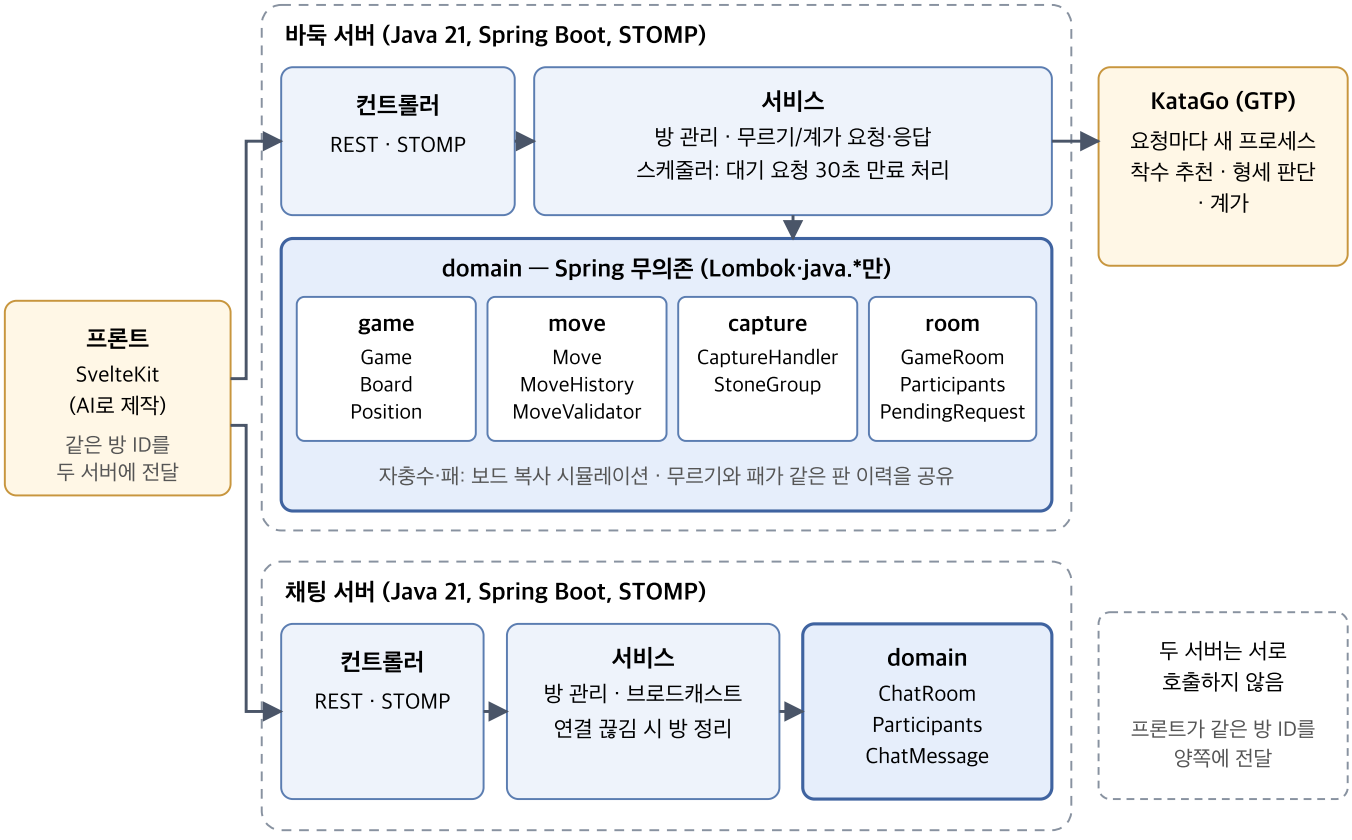
다음에 개선할 것 기본이 꺼진 보안 설정(서비스 토큰 검증 등)을 운영에서 강제 모드로 올리는 것, DB 마이그레이션까지 되돌리는 롤백, 로그인 방어 상태를 공유 저장소로 옮기는 것, 문서와 코드가 어긋난 부분 정리입니다.

4. 온라인 바둑 — 바둑 서버와 채팅 서버

2025-11~2026-02 · 우아한테크코스 프리코스 최종 과제 · 백엔드 2개 단독 · 프론트는 AI로 제작 · Java 21, Spring Boot 3.5, STOMP+SockJS, KataGo(GTP), Docker

목표 도메인을 엄밀하게 정의하고 그 도메인에 맞춰 코드를 짜는 데 중점을 두었습니다. "도메인을 어떻게 설계할 것인가"와 "어떤 기능이 있으면 사용자가 편할까"를 함께 고민했고, 후자의 답으로 바둑 AI(KataGo)의 착수 추천·형세 판단을 넣었습니다.

- **설계를 먼저 문서로**: 구현 전에 기능 목록(242줄)을 먼저 커밋했고 도메인 객체 10개와 메서드가 이미 정의되어 있었습니다. 이후 객체 단위로 "구현 + 테스트"를 쌓았고, 따낸 좌표를 따로 기록하던 방식을 버리고 판 이력 하나로 무르기와 패를 함께 처리하게 고친 이력도 남아 있습니다.
- **도메인**: 도메인 패키지는 Spring에 의존하지 않습니다. 좌표 `Position`은 값 객체로 1~19를 생성 시점에 검증하고, 자충수는 "상대를 먼저 따낼 수 있는가, 아니면 내 연결 그룹 활로가 0인가"로, 패는 "착수·따냄 뒤 국면이 직전 국면과 같은가"로 판정합니다. 과제의 에러 문구 4개를 코드와 일치시켰고, 실제 패 시나리오를 도메인 테스트와 HTTP 통합 테스트 두 곳에서 확인합니다. 바둑 서버 테스트 185개.
- **사용자 편의**: KataGo로 착수 추천과 형세 판단을 제공하고, 대국을 끝내지 않는 형세 판단과 끝내는 계가를 분리했습니다. 무르기·계가는 상대 동의를 받는 요청/응답이며 30초가 지나면 만료 처리하고, 연결이 끊기면 대기 요청을 취소하고 방을 정리합니다.
- **채팅 서버**: 입장·퇴장·종료 규칙을 도메인 객체가 검증하고(도메인 테스트 31개, 전체 54개), 탭을 닫아 비정상 종료돼도 방이 정리되는 경로를 직접 설계했습니다.



온라인 바둑 구조 (소스를 전부 읽고 그린 그림)

다음에 개선할 것 슈퍼코·패스·덤·종료 판정 구현, 방 상태를 불리언 조합이 아닌 상태 모델로 정리하는 것, 시작 전 착수와 동의 없이 동작하는 직접 엔드포인트 차단입니다.

그 밖의 프로젝트

- **KelogBackend** (2024-04~09, 2인): Velog형 블로그 백엔드. main 코드의 약 70%를 맡았고(알림·에러 코드 체계는 팀원), 목록 6종을 `findPosts` 하나로 공통화했습니다. 트렌딩 점수는 DB 서브쿼리로 계산해 6시간마다 캐시하고, 소유권 검사는 `@Identification` AOP로 분리했습니다. 테스트 123개.
- **wolMgr** (2023-03~06, 4인): Wake-on-LAN 관리 웹. 로그인·JWT 인가 필터, 회원-기기 N:M 권한 모델, 이메일 인증을 맡았습니다. WOL 패킷 전송과 기기별 권한 확인은 팀원 코드입니다.
- **team4** (2023-02, 6인): 커뮤니티 웹. 채팅을 맡아 STOMP 없이 순수 WebSocket으로 방별 브로드캐스트와 방 목록 동기화를 직접 구현했습니다.

AI 도구 활용: 구매대행 업무 자동화의 2단계(통합 운영 시스템), Doro, 온라인 바둑의 프론트는 AI 코딩 도구를 활용해 구현했고, 요구 정의·구조 설계·검증은 직접 했습니다. 나머지 프로젝트는 AI 없이 작성했습니다. 팀 프로젝트의 수치는 git blame으로 확인한 줄 비율이며 팀원 코드는 제 몫에서 뺐습니다.